

Proyecto Programado CI-0123
Armador de figuritas con piezas plásticas (Lego®)
Autores: Equipo 4-Grupo 3

Socios: Equipo 1 (BST)
Equipo 2 (Los teletubbies)
Equipo 3 (Brigada A)
Equipo 5 (Plan B)

Protocolo Intragrupal Uppercase v4.2

En este documento se explica el protocolo de comunicación entre los elementos del proyecto. Se tiene un cliente, un intermediario (fork) y un servidor. El cliente únicamente se comunica con el fork, quien a su vez se comunica con el servidor. El servidor se comunica con el fork, que transmite sus mensajes al cliente. Varios clientes y servidores se pueden conectar al mismo fork, por lo que este crea un hilo para manejar independientemente cada conexión ($C_1 - F - S_1$).

1. Descripción

El protocolo tiene 3 parámetros separados por el carácter /. Se llama “Uppercase” ya que sus dos primeros parámetros sólo utilizan una letra mayúscula para indicar el tipo de mensaje a transmitir. Los mensajes siguen una estructura estándar con el siguiente formato:

[Protocolo]/[Comando]/[mensaje]

Dónde *Protocolo* siempre es la letra P, su única función es indicar que se está utilizando este protocolo. *Comando* es otra letra mayúscula que indica la función a realizar (véase Tabla 1). El tercer parámetro es un mensaje opcional, se utiliza para enviar datos (directorios y figuras) o información (puerto y dirección del servidor). En algunos casos, como en un cierre de conexión, *mensaje* puede omitirse, ya que el comando por sí solo describe completamente la acción.

Como se tiene la restricción de ser 3 parámetros y de que los dos primeros sean solo una letra entonces se sabe que los primeros 4 caracteres son parte del protocolo y el resto es parte del mensaje. Esto permite que se pueda utilizar “P/” dentro del mensaje sin generar problemas.

Ventajas:

- Implementación: la construcción, procesamiento y limpieza del protocolo conllevan métodos fáciles de realizar para el programador de los elementos.
- User Friendly: por su sencillez, cualquier usuario puede aprender y utilizarlo rápidamente.

2. Comandos

A continuación se muestra una tabla con los comandos y una breve descripción.

Nombre	Representación	Descripción
ACK	A	Indica que el comando se hizo correctamente
Connect	C	Crear conexión inicial fork-server
Data	D	Respuesta del server (directorios, figuras)
Fork	F	Utilizado por los Forks para solicitar la lista de figuras de otro Fork (no la lista completa)
Get	G	Solicitud de figuras
Patch	P	Actualización de servidores
Quit	Q	Cerrar conexión fork-server
Request	R	Usado por el Cliente para pedirle a su Fork el listado completo de las figuras disponibles
Server Data	S	Dirección IP y puerto del servidor

Tabla 1. Comandos del Protocolo

3. Flujo de Conexión

Descubrimiento de Intermediario en otras Islas

Cada Intermediario tendrá 8 hilos, uno para descubrir a su propio servidor, 5 para descubrir otros intermediarios, uno para responder solicitudes de otros Intermediarios y uno para atender clientes. La implementación del descubrimiento de su propio servidor es independiente de cada grupo. Para el descubrimiento de los Intermediarios en otras islas se decidió el siguiente esquema: cada isla escucha si alguna otra le pregunta si está disponible y los otros 5 hilos pasan preguntando por disponibilidad.

Isla 1 escucha si alguien la necesita y pregunta por Isla 2, 3, 4, 5, 6.

Los hilos dedicados a hacer el sendTo debe realizarlo cada cierto tiempo (puede configurar un intervalo de tiempo) para permitir que cualquier elemento se pueda levantar primero.

Cada isla tiene asignada una dirección IP por la cual se realiza el broadcast, como es el mismo BC por el cual se descubre su propio servidor entonces deben utilizar puertos diferentes. Se decidió que los Intermediarios se comunicarán por medio del **puerto 2026**.

Ejemplo Isla 1 descubre a Isla 2

Cuando se levante Isla 1 escucha por BC(172.16.123.31:2026), habla por BC(.47 , .63 , .79 , .95 , .111).

Cuando se levante Isla 2 escucha por BC(172.16.123.47:2026), habla por BC(.31 , .63 , .79 , .95 , .111)

Como Isla 1 habla por una dirección donde escucha Isla 2 ocurrirá el siguiente intercambio

Isla 1 (habla por .47:2026) > P/F/ > Isla 2 (escucha por .47:2026)

Isla 2 > P/D/fig1,fig2,fig3... > Isla 1

Por medio de estos mensajes UDP puede conseguir la dirección de origen del intermediario, su listado de figuras y así construir su mapa. La implementación de este mapa es individual para cada grupo.

Con el comando P/F/ se pregunta por las figuras específicas de su isla por lo que su intermediario debe tener alguna manera de identificar las figuras provenientes de su propio servidor y aquellas que vienen de otros servidores. Se trabaja de esta manera para evitar que se envíen figuras repetidas. Además tiene que seguir el formato de nombres separados con una coma sin índices, saltos de línea o algún otro estilo. El comando P/R/ si envía el listado completo de figuras.

Por lo tanto, su intermediario no anunciará que está disponible apenas se ejecuta, empieza a anunciar cuando tenga las figuras de su servidor.

Para no tener que enviar un mensaje adicional donde el intermediario indica cuál es su puerto entonces utilizará el mismo que para el broadcast. Es decir todo fork atiende en el puerto 2026.

Comunicación con Cliente

El cliente conoce la dirección IP y puerto del intermediario, cuando inicia establece una conexión TCP. El cliente puede digitar comandos siguiendo el protocolo establecido, si digita otra cosa el programa le indicará que cometió un error. El cliente puede solicitar el directorio de figuras (REQUEST), una figura en particular (GET) o terminar la conexión (QUIT).

Una vez tiene el listado de figuras, el cliente puede solicitar alguna. El intermediario recibe la solicitud, como tiene un mapa de servidores con figuras lo utiliza para identificar el servidor que tiene esta figura, establece una conexión TCP solo con ese servidor y reenvía la solicitud. Si durante este tiempo el servidor actualizó sus figuras y ahora no cuenta con la que fue solicitada responderá con un mensaje de error. De lo contrario responde con la figura. Intercambio:

Client > P/G/fig1 > Fork

Fork > P/G/fig1 > Server

Server > P/D/brown block 1, red block... > Fork
Fork > P/D/brown block 1, red block > Client

El cliente recibe la figura, vuelve al menú. Para pedir otra figura u otra vez el directorio disponible repite el proceso anterior. Para terminar la conexión utiliza el comando de QUIT, cierra el socket conectado al intermediario y termina el programa. Este comando también se utiliza para cerrar las conexiones TCP entre intermediario y servidor cada vez que se hace un intercambio de mensajes entre ellos.

Client > P/Q/ > Fork

Comunicación con Cliente Web

La comunicación entre el Cliente y el Intermediario de su propia Isla puede ser por medio de HTTP. Sin embargo, la comunicación entre intermediarios debe seguir el protocolo, es decir, su intermediario obtiene la solicitud del cliente la convierte en el protocolo y luego se la envía al servidor o intermediario correspondiente. Su intermediario recibe la respuesta con el protocolo, extrae la información necesaria y la convierte a su salida HTTP para contestarle al cliente.

De esta forma no arrastramos un mensaje en HTTP entre islas y así cada equipo puede configurar su web de la forma que desee sin afectar a las demás.

- Isla 1: 172.16.123.[19-30] > BC 172.16.123.31
- Isla 2: 172.16.123.[35-46] > BC 172.16.123.47
- Isla 3: 172.16.123.[51-62] > BC 172.16.123.63
- Isla 4: 172.16.123.[67-78] > BC 172.16.123.79
- Isla 5: 172.16.123.[83-94] > BC 172.16.123.95
- Isla 6: 172.16.123.[99-110] > BC 172.16.123.111

Muerte de Servidores

Cuando un servidor se cae no lo reporta. En el momento que el fork intente hacer un *connect* en esa dirección y puerto fallará, esto significa que el servidor ya no está disponible por lo que simplemente elimina los datos de su tabla de enrutamiento. La próxima vez que un cliente solicite el listado de figuras ya no aparecerán las de este servidor, pues fue borrado. Se reactiva el hilo secundario que hace broadcast a la dirección de la isla donde estaba ese servidor esperando que se llegue a habilitar otro.

4. Posibles Eventos-Bitácoras

Server se levanta antes que el Fork: Server se quedará esperando por el mensaje en Broadcast.

Client se levanta antes que el Fork: a la hora de intentar establecer la conexión con el Fork el socket reporta error y se termina el programa.

Client se levanta antes que algún Server: cuando el cliente le pide al Fork la lista de figuras disponibles este le responde con “P/D/No servers available” lo que le indica que todavía no tiene ninguna.

Client solicita una figura que no existe: Fork responde con un mensaje de error 404.

Client solicita una figura que ha sido borrada: Server responde con un mensaje de error 404.

Client utiliza REQUEST incorrectamente: el comando de REQUEST sirve para pedir el directorio de figuras de los servidores disponibles. Si solicita otra cosa, por ejemplo una figura, el servidor le responderá con una advertencia.

Server se cierra durante la conexión: cuando el cliente envía una solicitud, el Fork la recibe, detecta que el Server ya no está y le responde al cliente “P/D/Server not found”.

Fork se cierra durante la conexión: cuando el cliente envía una solicitud el socket reporta error y el programa termina.

Client se cierra sin hacer QUIT: el cliente termina de golpe, por ejemplo con un ctrl+C, se contemplan 2 posibles alternativas de solución a desarrollar:

1. Hacer que el programa detecte si termina de golpe y enviar un QUIT automático.
2. Tanto Server como Fork tienen un límite de tiempo de espera.